

The Crossover Payload of an Encrypted UDP Data Plane

Qiyue Zhao

Abstract

We show that the per-packet cost of an encrypted UDP data plane splits into a fixed term a and a per-byte term b whose ratio, the crossover payload $s^* = a/b$, is the one quantity that transfers between machines when neither term alone does. Firstly, we fit that model over 104 configurations covering four cipher implementations, the hardware crypto instructions present and masked, and the whole payload range an MTU allows. Secondly, we measure what the system-call boundary contributes to a directly, from a submission path that issues no system calls, instead of extrapolating it from a batching curve. Moreover, we apply the same fixed-versus-marginal split at two further boundaries: the isolation layer around a virtual network function, and the bus to a GPU. Finally, every control states its prediction before it runs, and every number in this text is generated from the dataset rather than typed into it. On the host measured here `lr-aes-256-gcm` gives $a = 6\,768$ cycles and $b = 1.869$ cycles per payload byte, so $s^* = 3\,621$ bytes against a largest cariable payload of 1 432 bytes, and the fixed cost therefore dominates at every size this data plane can send. Against `OpenSSL`, the implementation written for this study holds the fixed term to a factor of 0.85 at 1.85 times the per-byte cost, and 1 642 of its 7 982 cycles of fixed cost are the AEAD's own.

Keywords

data plane, authenticated encryption, cost model, measurement

1 Introduction

A tunnel endpoint that terminates one security association and originates another pays two kinds of cost for every packet it forwards. One kind does not depend on how large the packet is: the system calls that move it across the kernel boundary, the header parse, the anti-replay check of ESP and its descendants [7], the nonce derivation, and the cipher's own per-message setup and teardown. The other kind is proportional to the payload, and covers the bulk encryption and decryption together with the memory traffic that feeds them.

Writing the first as a cycles and the second as b cycles per payload byte gives

$$C(s) = a + b s, \quad s^* = \frac{a}{b}, \quad (1)$$

where s^* is the payload at which the two halves are equal. Below s^* a packet costs mostly what it costs to be a packet at all; above it, mostly what it costs to carry that many bytes. The two sides call for opposite work. Below s^* , the useful optimisations are batching and the removal of system calls. Above it, they are a faster cipher or fewer passes over the payload.

The ratio is more portable than either term. Both a and b are counts of cycles, and both scale with the clock, so s^* survives clock scaling and can be compared across machines whose absolute numbers cannot. That portability is why this study reports it.

What is measured here. The data plane is a decrypt-then-re-encrypt forwarding element, so every payload byte crosses the

cipher twice. Crossing twice makes b large enough to separate cleanly from a , and the report states the factor of two rather than folding it away. Four factors are varied against it: the cipher, in four levels; the presence of the hardware crypto instructions; the transport, in four levels whose system-call rates differ by construction; and the payload size, over a grid dense at the small end where a dominates.

Two implementations of each cipher. For `AES-256-GCM` [4] and `ChaCha20-Poly1305` [11] we measure `OpenSSL`'s implementation and one written for this study. Ours uses `AES-NI` with a `PCLMULQDQ GHASH` [5] and a stitched single-pass counter-mode-and-hash inner loop [8] on x86-64, ARMv8 AES with `PMULL` on aarch64, and `ChaCha20` vectorised with `AVX2` or `NEON`. Both agree with `OpenSSL` byte for byte at every length in the sweep, which the self-test checks before any timing run. A second implementation is what separates "this cipher costs x " from "this library's version of this cipher costs x ", and the gap between the two is reported rather than hidden.

Contributions.

- A measured cost model for an encrypted UDP data plane, with the crossover payload s^* and its confidence interval, for four cipher implementations with hardware crypto present and masked.
- A direct measurement of what the system-call boundary contributes to a , taken from a submission path that issues no system calls [1] rather than extrapolated from a batching curve.
- The same fixed-versus-marginal decomposition applied to two further boundaries: the isolation layer around a virtual network function, and the bus between a CPU and a GPU.
- An artefact that refuses to produce a measurement on a host that cannot hold one, records why every unavailable experiment was unavailable, and generates every number in this text from the dataset.

2 Method

The measured path. Every experiment times the same loop, so that a difference between two runs is attributable to the factor that was varied and not to a different amount of work:

receive → replay check → open
→ rekey check → seal → send.

The replay check precedes the cipher deliberately. A replayed packet must not cost a decryption, which is also why the window's cost lands in a .

Packets carry a 24-byte header and a 16-byte authentication tag. The nonce is derived from the epoch and sequence number through the key schedule rather than carried, as deployed protocols do, and that is what keeps the header at 24 bytes. The header is authenticated as associated data, so a tampered sequence number, epoch or timestamp is rejected along with a tampered payload.

Table 1: The measurement host, as recorded with every dataset fragment.

property	value
CPU	Intel(R) Xeon(R) Platinum 8481C CPU @ 2.70GHz
architecture	x86_64
physical cores	2
SMT	on
kernel	Linux 6.17.0-1022-gcp
counter	calibrated-vs-CLOCK_MONOTONIC_RAW at 2.700 GHz, invariant
governor	unavailable-in-guest
kernel isolation	none
interface	ens3 (gve)
isolation tier	host
OpenSSL	OpenSSL 3.0.13 30 Jan 2024
compiler	GNU 13.3.0
flags	-O3 -march=native -fno-omit-frame-pointer
host class	constrained

With 28 bytes of IP and UDP headers, the largest payload that fits a 1 500-byte MTU is 1 432 bytes, and that is the top of the sweep.

Statistical discipline. The dataset assembled for this report contains 11 fragments. The experiments absent from it are `e2_mpi`.

Each point is measured 11 times. The replicate count is odd so that the median is an observed value rather than an average of two. Spread is reported as a percentile-method bootstrap interval on the median rather than a standard deviation, because the distribution of a per-packet cost is right-skewed by scheduling events and a symmetric interval would misrepresent it [6].

A point whose coefficient of variation exceeds 10% is marked unstable. It is recorded rather than dropped: excluded points appear hollow in the figures, so a reader can see how far they sat from the line.

The sweep order is interleaved and seeded rather than nested. A nested loop confounds the factor with time. If the machine warms up, or another tenant arrives halfway through, the whole effect lands on whichever level was running then. Each replicate of each unit is instead a separate entry in one list, shuffled with a fixed seed that is recorded with the data.

An anchor configuration is measured before and after every sweep, as the median of 5 units at each end rather than as a single observation. An anchor built from two single measurements carries the same replicate variance as any other point and would void sweeps on its own noise. If the two ends disagree by more than ten per cent the sweep is marked void rather than reported. In the run described here the anchor moved by 0.26%, so the sweep is valid.

Varying the hardware crypto factor. The capability mask that clears AES-NI and the carry-less multiply is read by `libcrypto` when it loads, and by the dispatcher of the implementations written here for the same reason. It cannot be changed inside a running process, so each unit of the sweep runs in a fresh child process with the mask set or cleared. The child reports back whether the mask took effect, and a unit whose child disagrees with the request is discarded. Without that check, a mask that silently failed would appear in the data as a null result.

Applying the mask to both implementations matters. Masking only OpenSSL would compare its software path against our hardware path, which measures nothing.

Host classification. A host that cannot hold a measurement is not permitted to write one. Three classes are enforced in code: *development*, which builds, self-tests and typesets but never measures; *constrained*, which measures and attaches its caveats to every dataset; and *measurement*, which adds kernel isolation on the data-plane cores and at least two homogeneous cores. The host used here is a 2-core x86_64 machine (Intel(R) Xeon(R) Platinum 8481C CPU @ 2.70GHz) running Linux 6.17.0-1022-gcp, with an invariant counter at 2.700 GHz, built by GNU 13.3.0 against OpenSSL 3.0.13 30 Jan 2024. It classifies as constrained in isolation tier host. Table 1 gives its full description, which travels with every fragment of the dataset.

Topology. Where a second host is available the load generator runs on it and sends across a physical interface; the data plane binds that interface and forwards re-encrypted packets back. The generator is driven for a whole sweep over one control connection rather than over per-unit ssh, which would spend more time on ssh than on measurement. Without a second host the generator is a thread on the same machine and the packets never leave it. That fallback is recorded in every affected point, and its consequences are stated where they matter, in Section 5.

3 The cost model and the crossover payload

The sweep comprises 104 distinct configurations. Figure 1 shows it and Table 2 the fits. The relationship is linear over the whole payload range for every cell but one. For `lr-aes-256-gcm` with the hardware instructions masked, a quadratic term is significant at the 1% level, so the linear model is an approximation there and a and b should be read as fitted over this payload range rather than as properties of the code.

The fits are weighted by the inverse variance of each point rather than left unweighted. Small payloads are noisier, because that is where a dominates and where scheduling noise lands, and an unweighted fit would let the noisiest points set the intercept, which is the parameter the whole study turns on. The interval on s^* is propagated through the covariance of a and b rather than obtained by dividing their separate intervals. The two are strongly anticorrelated, since a fit that raises the intercept lowers the slope, and ignoring that inflates the interval several-fold.

The headline. With hardware crypto available, `lr-aes-256-gcm` has $a = 6\,768$ cycles and $b = 1.869$ cycles per payload byte, giving $s^* = 3\,621$ bytes. The largest payload this data plane can carry inside an MTU is 1 432 bytes, so the crossover lies outside the reachable range by a factor of more than two. The fixed cost therefore dominates at every payload size this configuration can send, and no improvement to the cipher can change its throughput by much: the packet costs, not the byte. Where s^* falls inside the range instead, as it does for `lr-chacha20-poly1305` in Table 2, the crossover is reachable and the choice of cipher starts to matter above it.

Reading a across experiments. Table 2 obtains a from a fit across the whole payload sweep, while Table 7 and Table 6 report a cost measured at one configuration. The three are separate experiments on the same host and their values for the same cipher agree to about one per cent, which is the reproducibility of the harness rather than a discrepancy. Comparisons in this report are always drawn within one experiment.

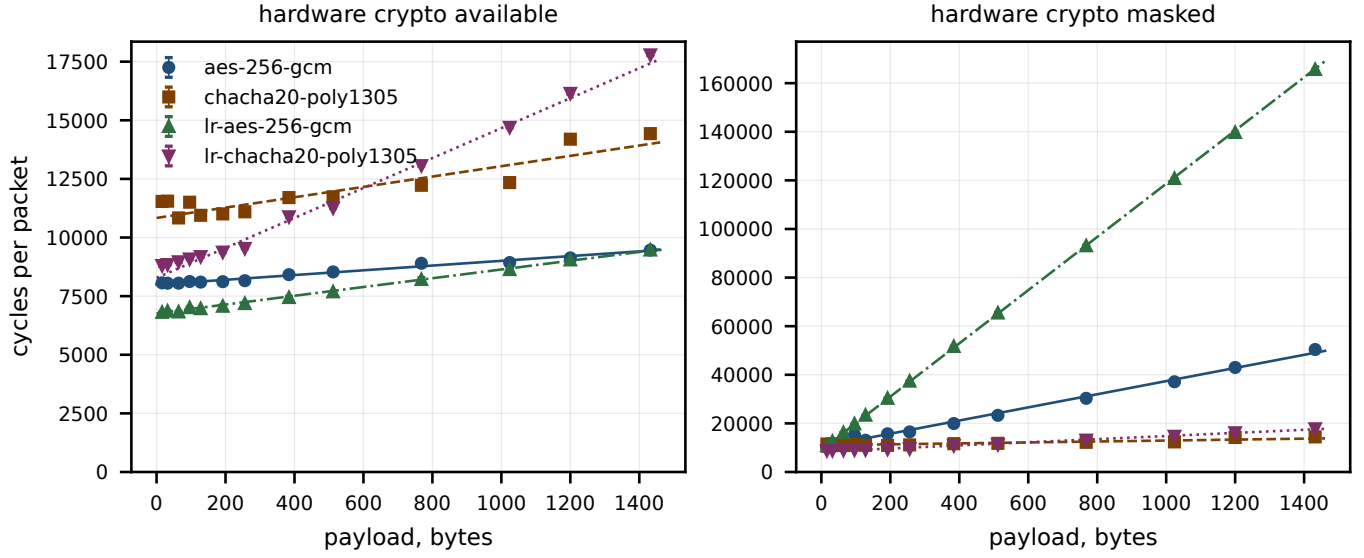


Figure 1: Cycles per packet against payload size, with the fitted $C(s) = a + bs$. Bars are bootstrap intervals on the median of 11 replicates; hollow markers are points the stability gate excluded from the fit. Left, the hardware crypto instructions available; right, the same measurement with them masked in both implementations.

Table 2: Fitted cost model. a is the payload-independent cost, b the cost per payload byte over a path that crosses the cipher twice, and $s^* = a/b$ the crossover payload with its 95% interval propagated through the parameter covariance.

cipher	hw	a (cyc)	b (cyc/B)	s^* (B)	95% CI	R^2
aes-256-gcm	yes	7 995	1.011	7 906	[7 102, 8 709]	0.9791
aes-256-gcm	no	10 363	27.033	383	[338, 429]	0.9926
chacha20-poly1305	yes	10 835	2.206	4 911	[3 339, 6 483]	0.8348
chacha20-poly1305	no	10 996	1.943	5 659	[3 480, 7 838]	0.7735
lr-aes-256-gcm	yes	6 768	1.869	3 621	[3 505, 3 736]	0.9980
lr-aes-256-gcm	no	9 121	109.495	83	[82, 84]	0.9999
lr-chacha20-poly1305	yes	8 279	6.387	1 296	[1 165, 1 427]	0.9874
lr-chacha20-poly1305	no	8 382	6.416	1 307	[1 219, 1 394]	0.9953

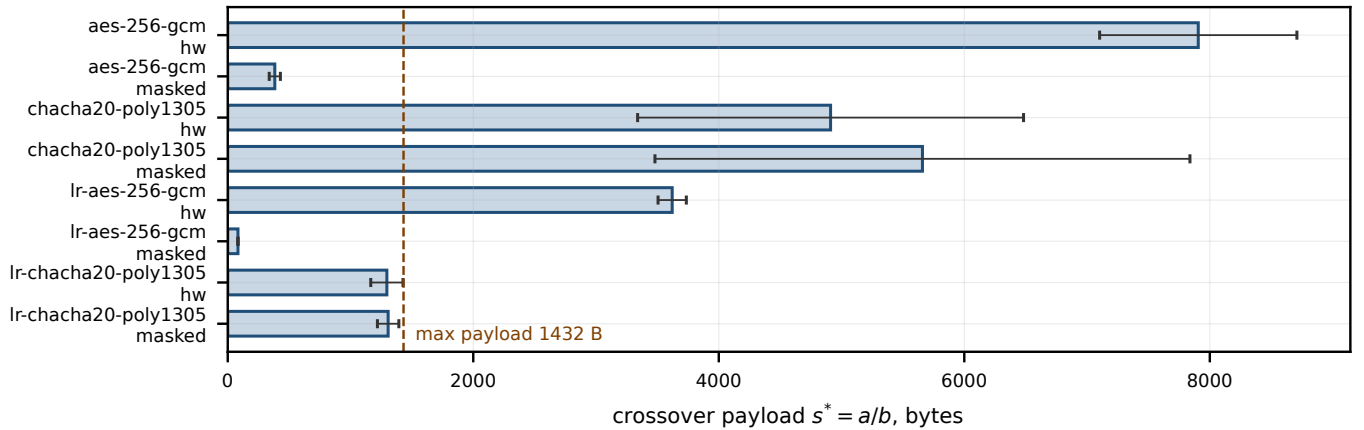


Figure 2: The crossover payload for each cell of the design, with its 95% interval. The dashed line is the largest payload this data plane can carry inside an MTU: a crossover to the right of it means the fixed cost dominates at every size the data plane can send.

Our implementations against OpenSSL’s. The AES-256-GCM written for this study reaches b a factor of 1.85 of OpenSSL’s, with a fixed cost a factor of 0.85 of it. Where that factor exceeds one it has a specific cause, and we state it rather than attributing it to implementation quality in general. OpenSSL’s assembly interleaves eight AES blocks using the vector AES extensions, while the implementation here interleaves four using the 128-bit instructions. The stitched inner loop, which computes counter-mode encryption and the GHASH in one pass over the data rather than two, recovers most but not all of that difference. Our implementation is checked byte for byte against OpenSSL at every length in the sweep, so the residual is a property of the instruction selection and not of correctness.

The comparison is useful in both directions. It bounds how much of the measured b is intrinsic to the algorithm rather than to one library, and it puts a number on what the wider vector path buys, which is invisible when only one implementation is measured.

Masking the accelerator. The masked arm is not a hypothetical. It is what the same code does on a machine without the instructions, which is what an embedded or older deployment target looks like. The change in b between the two arms is the whole contribution of the hardware accelerator to the bulk term, and Table 2 shows it spanning more than an order of magnitude. The change in a is smaller. It is the accelerator’s contribution to the cipher’s per-message setup, the part that does not scale with payload and therefore does not go away by sending larger packets.

4 Where the fixed cost lives: the kernel boundary, the shards and the memory roof

Four transports carry the same hot path. `posix` issues one receive and one send per packet. `mmsg` issues one of each per batch. `uring` batches submissions through a shared ring [1]. `uring-sqpoll` adds a kernel-side submission poller, so submission costs no system call at all. That is as close to a kernel-bypass data path [2, 12] as an unprivileged process reaches without a userspace driver.

The last of those is the point of the experiment. Without it, the system-call share of a can only be extrapolated from the shape of the batching curve. With it, the quantity is measured: at batch depth one, `posix` costs 9 392 cycles per packet and `uring-sqpoll` costs 30 935, a difference of -21 543 cycles.

When that difference is not a bound. A submission poller is a second runnable thread. On a host with fewer usable cores than the poller needs, it competes with the data plane for the same core, and the calls it removes cost more in scheduling than they saved. The harness detects and records that condition rather than inferring it, because a negative saving is a property of the host and not of the mechanism, and presenting it as either a bug or a benefit would be wrong. The dataset records the condition as follows: the poller has a core of its own, so the difference between `posix` and `uring-sqpoll` at $B = 1$ bounds the cost of the system-call boundary. The measured difference is nevertheless negative, so on this run the polled path bounds nothing. What it measures is the poller competing for a machine with 2 physical cores, and Table 3 shows `uring-sqpoll` approaching the other backends only at the deepest batch in the grid.

Table 3: Transport grid at payload 1 432 bytes.

backend	B	cycles/pkt	syscalls/pkt	pkt/s
mmsg	1	10 267	2.000	262 982
mmsg	2	9 312	1.000	289 935
mmsg	4	8 805	0.500	306 641
mmsg	8	8 576	0.250	314 830
mmsg	16	8 408	0.125	321 135
mmsg	32	8 309	0.063	324 960
mmsg	64	8 234	0.031	327 907
posix	1	9 392	2.000	287 486
uring	1	11 291	3.000	239 133
uring	2	10 032	2.000	269 150
uring	4	9 417	1.500	286 719
uring	8	9 097	1.250	296 803
uring	16	8 918	1.125	302 768
uring	32	8 789	1.063	307 210
uring	64	8 734	1.031	309 141
uring-sqpoll	1	30 935	0.000	87 280
uring-sqpoll	2	19 992	0.000	135 056
uring-sqpoll	4	14 790	0.000	182 558
uring-sqpoll	8	11 810	0.000	228 619
uring-sqpoll	16	10 835	0.000	249 184
uring-sqpoll	32	10 329	0.000	261 399
uring-sqpoll	64	10 017	0.000	269 542

Batching. Table 3 gives the full grid. The batched backends converge as the depth grows, which is expected: past the point where the amortised system call is small next to the cipher, the remaining differences are in how each ring reports completions. On this host the best batched configuration was `mmsg` at 8 234 cycles per packet.

The measured syscall counts are worth reading alongside the costs. They confirm that each backend does what its design says: two calls per packet for `posix`, $2/B$ for `mmsg`, and none on the submission side for `uring-sqpoll`. That is the check that the configuration under measurement is the configuration intended.

Widening the data plane. Each shard owns its socket, key schedule, replay window and arena, and shares nothing mutable with any other. The scaling question is therefore not whether a lock contends, but what the machine does when n identical independent pipelines run at once: last-level cache, memory bandwidth, and on a hyper-threaded host the sharing of one physical core’s execution ports.

Two laws are reported, because they answer different questions and quoting one for the other is a standard way to overstate a result. *Strong* scaling fixes the total packet count and divides it, and answers whether adding a core helps with a given amount of traffic. *Weak* scaling gives each shard the same count whatever n is, and answers whether the machine can absorb n times the traffic with n times the cores.

The worker pool is expressed with OpenMP where the compiler provides it and with `std::thread` otherwise; on this run it was `openmp`. Threads are created and shards constructed and warmed before the timed region opens, and all workers wait at a barrier, so that thread creation is not charged to the data plane and no worker measures an uncontended machine because it started first.

Fitting Amdahl’s law to the strong-scaling curve gives a serial fraction of 0.1012 over 2 shards. Table 4 gives the points. The cross-node half of this experiment, which runs one MPI rank per node to see whether the per-packet cost stays put when the ranks are

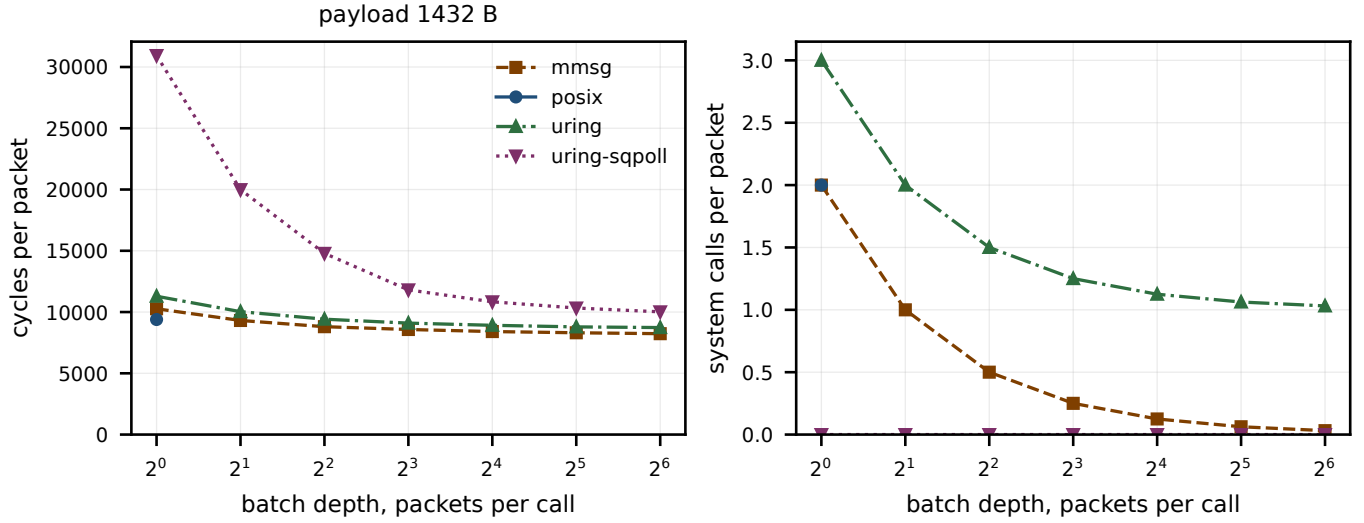


Figure 3: Per-packet cost and measured system calls per packet against batch depth, at payload 1432 bytes. Syscall counts are counted inside each backend rather than derived from the design, because a batch that returns short does not issue the number of calls the design intends.

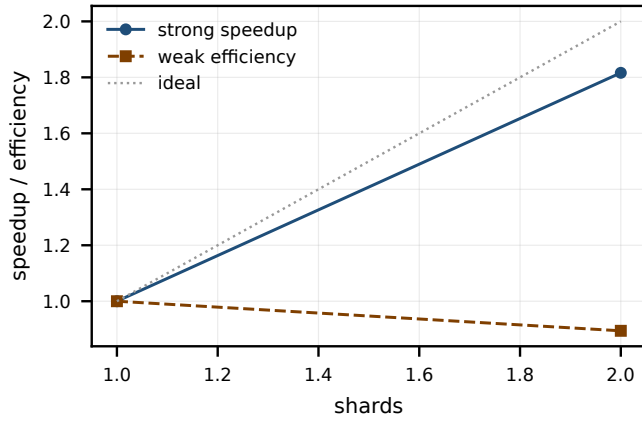


Figure 4: Strong speedup and weak efficiency against the number of shards. Shards share nothing mutable, so what this measures is contention for the memory system rather than for a lock.

Table 4: Scaling of the shard pool.

shards	strong speedup	strong eff.	weak eff.	cycles/pkt
1	1.000	1.000	1.000	8 488
2	1.816	0.908	0.894	9 345

on separate hosts, did not run here: its fragment is the one missing from the dataset, and Section 8 says what that costs the argument. **The memory roof.** A data plane cannot move payload bytes faster than the memory system supplies them, divided by the number of times each byte is touched. A STREAM triad [10] over a geometric range of working sets measures the numerator directly, so the

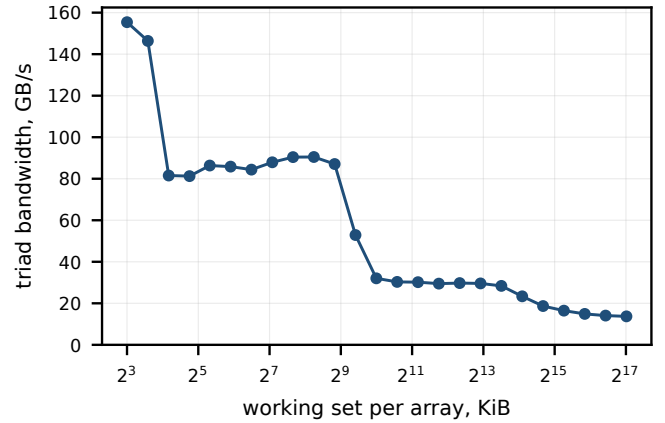


Figure 5: Triad bandwidth against working-set size. The plateaus are the cache hierarchy; the rightmost is the sustained figure the roof is computed from.

scaling curve above can be read against a roof [13] rather than against nothing.

The largest working set measured sustains 13.7 GB/s. Each payload byte makes 8 passes through memory in the hot path: the kernel copy in on receive, the read and write of the decryption, the read and write of the re-encryption, and the kernel copy out on send, each counted as one read and one write. Dividing gives a roof of 1.71 GB/s of payload, the rate this data plane cannot exceed on this machine however fast the cipher becomes. Figure 5 shows the plateaus of the cache hierarchy that produce that number, and therefore which level a packet buffer of a given size is served from.

5 Latency under load

A service cost implies a capacity, and a capacity implies a knee: below it delay is service time, above it delay is queue and grows until something drops. The offered rate is swept across that knee.

The load must be open-loop. A closed-loop generator waits for the previous packet to be served before sending the next, so it cannot offer more than the receiver’s capacity, and the queueing this experiment exists to expose never forms. Reporting a closed-loop curve as a latency-under-load result is a standard error, and it always produces a flatteringly flat tail.

What was measured, and on which clock. In the two-host topology, latency is a *round trip* measured entirely on the generator’s own clock. The generator stamps the packet, the data plane forwards it back, and the generator reads the stamp out of the header, which travels in the clear as associated data and so needs no decryption. No clock synchronisation between the hosts is assumed and no one-way delay is claimed. The no-crypto echo baseline, measured the same way, is what makes the difference interpretable. This run used the wire topology under network condition multiple, with the generator on a separate host: generator on a separate host across the measured link.

What this run produced. The capacity used to set the rate grid is measured on the same configuration immediately beforehand rather than taken as a constant, so that the grid straddles the knee on whatever machine this is. Here that capacity was 311 881 packets per second. The rate sweep that follows it returned no packets over the wire, so the knee is reported as 0 packets per second, or 0.0% of capacity, and no curve is drawn. That is a recorded absence rather than a measured flat tail: the harness marks every point of the sweep unavailable with its reason, and the analysis declines to read a knee from it. Section 8 states what the missing curve costs the rest of the argument.

6 Independent references, and the same question at two further boundaries

A cost model fitted to one program’s own measurements is self-consistent by construction. Independent references are placed beside it, and Figure 6 shows the per-byte comparison.

`openssl speed` times the same cipher with no packet framing at all, on the same machine, using a program this project did not write. It estimates b directly and independently, and reports 0.2302 cycles per byte against 0.4455 for this data plane per pass through the cipher. A disagreement here would mean the fit was wrong; agreement is the check that it is not.

`iperf3` over UDP measures the same interface with no cryptography and no user-space forwarding, giving the upper bound on what any encrypted data plane on this link could reach: 6.76 Gb/s against 3.32 Gb/s for this data plane. The third reference, a production kernel tunnel [3] carrying the same traffic over the same link, is the comparison a reader actually wants: not “is this fast” but “how does its per-packet cost compare with a deployed implementation of the same idea”. That reference did not run on this host, so Table 5 is absent from this build. The two implementations would in any case differ in ways the table would have to record, since a kernel tunnel does not cross the user-kernel boundary per packet and this

data plane decrypts and re-encrypts where a tunnel endpoint does one of the two.

The isolation boundary. The same decomposition applies to a second boundary. A virtual network function runs inside an isolation tier, and the expectation is that each tier adds fixed cost per packet without adding per-byte cost, in the same shape as the system call, one level out [9].

Four tiers are in the design: the virtual machine’s own kernel with its para-virtualised interface (gve), a network namespace joined by a veth pair, a runc container behind a bridge with its network address translation, and a gVisor sandbox in which system calls are serviced by a user-space kernel rather than by the host [14]. The last tier is where a should move most, since every call in the hot path then crosses into another process.

Only 2 tiers were measurable on this host, and the expected ordering did not appear in them. Relative to the host tier, the namespace tier measured 0.868, which is below it rather than above. Table 6 gives the absolute figures. A tier that measures cheaper than the host it sits on is not a saving from isolation. It is the distance between two single-configuration measurements on a shared machine, and it is the reason this report does not quote a per-tier penalty. What the two tiers do agree on is the syscall count, which is identical across them, so the difference is not a change in the number of boundary crossings.

This is not the cost of virtualisation. True bare metal is not obtainable on a public cloud, and the host tier is itself a hypervisor guest. What is reported is the *incremental* cost of each isolation layer above the virtual machine, not the absolute cost of virtualising a data plane. Stating that plainly matters more than the number, because a reader who took these figures as the cost of virtualisation would understate it by however much the hypervisor already costs, which this setup cannot see.

Chaining. A service function chain composes these endpoints. Each hop terminates one security association and originates another, which is what a chain of encrypting functions does, and the chain is built from veth-joined namespaces at lengths one through four. The slope of cost against chain length gives 4 cycles per hop. That is the price of composition, separate from the price of the function itself, and at this size it sits at the noise floor of the measurement rather than above it.

The same question at the bus. A GPU offload has the shape of Equation 1 with different constants:

$$T(n, s) = a_{\text{gpu}} + \frac{s n}{B_{\text{pcie}}} + \frac{s n}{R_{\text{gpu}}}, \quad (2)$$

where a_{gpu} is launch and synchronisation, the middle term is the transfer the CPU path does not pay at all, and the last is the kernel itself. The offload wins only when the batch is large enough to amortise a_{gpu} and the far side of the bus beats the CPU per byte. The middle term makes the second condition much harder than it looks, because bus bandwidth sits roughly an order of magnitude below what AES-NI sustains per core, so for a streaming data plane the bus is usually the answer before the kernel runs. The measurement host carried no GPU and no CUDA toolkit, so the constants of Equation 2 are not measured here and the dataset records the reason.

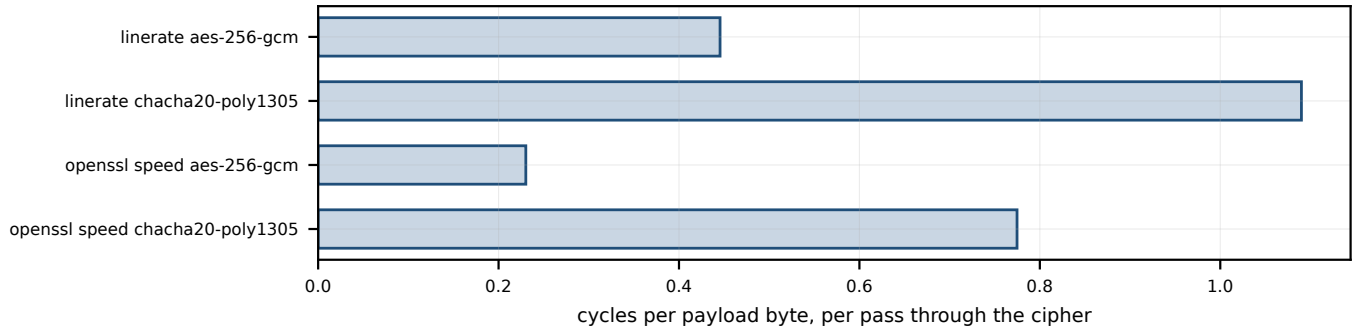


Figure 6: Per-byte cost of this data plane beside independent references. The data plane crosses each payload byte twice, so its per-byte cost is divided by two before being placed on the same axis.

Table 5: Independent references. The passes column is the number of times each payload byte crosses a cipher, which is what makes the per-byte figures comparable.

This table could not be generated from the dataset.

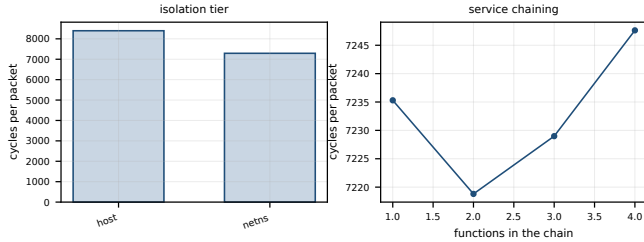


Figure 7: Per-packet cost across isolation tiers, and along a chain of encrypting virtual network functions.

Table 6: Isolation tiers. The relative column is against the host tier, which is the only comparison that transfers: the absolute cycle count is a property of this machine.

tier	driver	cycles/pkt	vs. host	syscalls/pkt
host	gve	8 397	1.000	2.000
netns	unknown	7 291	0.868	2.000

7 Controls

Each control carries a prediction made before the measurement and recorded next to the outcome. A control whose prediction is written down afterwards cannot fail, and a control that cannot fail is decoration. Of 5 controls that ran, 5 passed. Table 7 gives each one with its prediction and what happened.

The null cipher. Replacing the AEAD with a copy of the same length leaves everything else in the hot path unchanged: the same system calls, the same replay window, the same header parse, the same buffers. What remains is the fixed cost of the data plane with the cryptography removed, 6 340 cycles against 7 982 with it. The difference, 1 642 cycles, is what the AEAD contributes to a . It is per-message setup and teardown, it does not scale with payload, and it therefore cannot be amortised by sending larger packets.

Without this control, a would be a single opaque number; with it, it decomposes.

Whether the per-byte terms of the two configurations can be compared at all depends on the host. Where the fixed cost dwarfs the per-byte term across the whole payload range, two small slopes differ by less than the fit resolves, and requiring one to fall below the other would make this control fail on noise. The control measures that condition rather than assuming it. On this host the per-byte terms are separately identifiable: yes.

The masking control. If clearing the hardware crypto capability bits did not raise b by a large factor, the mask never reached the implementation, and every masked measurement in Section 3 would be void. This is the control that makes the masked arm trustworthy rather than merely present.

Instrumentation controls. Three checks confirm that the harness is not measuring itself. The cost of polling an empty socket is weighted by how often that actually happens in a run, because a poll that costs a tenth of a packet but occurs once per thousand packets contaminates nothing; the amortised figure is the one with a threshold attached. A forged packet is required to cost about what a valid one costs, so that rejection is neither a cheap path through the loop nor a side channel. And every cipher level in the design is confirmed constructible, since a missing level would silently shrink the experiment.

8 Threats to validity, and what this run could not measure

The binaries are tuned to this machine. Compilation uses the native architecture flags, so absolute cycle counts belong to the microarchitecture they were measured on. Portable binaries would understate what a tuned data plane achieves, which is the wrong error for this study to make. The crossover s^* is a ratio and transfers; a and b separately do not.

The host is shared and virtualised. The measurement host is classified constrained, and every caveat that classification implies travels with the dataset. Where kernel isolation on the data-plane

Table 7: Controls, with the prediction each was given before it ran.

control	verdict	outcome
null cipher removes the cryptographic cost	pass	null $a=6339.7$ $b=0.2483$ against aead $a=7982.0$ $b=0.9736$; the difference in a , 1642.3 cycles, is the AEAD's own per-packet cost. The per-byte terms are separately identifiable here and were compared.
masking hardware crypto raises the per-byte cost	pass	masked $b=27.5380$ against unmasked $b=0.9736$, a factor of 28.3; masked $a=9940.5$ against unmasked $a=7982.0$
the harness's own idle polling does not contaminate a measurement	pass	686.4 cycles per empty poll, 0.00000 empty polls per packet, so 0.000 cycles charged to each of 6339.7 cycles of fixed cost
a forged packet costs about what a valid one costs	pass	forged 780.4 cycles against valid 777.5 cycles, ratio 1.004
every cipher level in the design is constructible	pass	all four levels available

cores is absent, another tenant's work appears as variance rather than as bias. That is what the interleaved sweep order and the anchor check are for: drift that would otherwise land on one factor level is spread across all of them, and a machine that changed under the measurement voids the sweep instead of reporting it.

The path crosses each byte twice. The data plane decrypts and re-encrypts. That is a real forwarding topology and it makes b large enough to separate cleanly from a , but it is not the only topology. A one-pass endpoint would have roughly half this b and therefore roughly twice this s^* . The per-pass figures are reported alongside so that a reader can make that adjustment rather than guess at it.

The bound on kernel bypass is conditional. A submission poller that shares a core with the data plane is a competitor, not a bypass. Where that condition holds, the difference between the polled and unpolled paths bounds nothing, and Section 4 reports the condition rather than the bound.

Bare metal is not available. The virtualisation tiers are measured above a hypervisor guest, so they give the incremental cost of each isolation layer and not the cost of virtualisation itself. On this run two of the four tiers did not build, and the two that did ordered themselves the wrong way round, so no per-tier penalty is claimed.

Three experiments did not run here. The cross-node scaling fragment, the latency curve and the GPU offload are all absent, and each is recorded as unavailable with its reason rather than omitted. The consequences are specific. Without the cross-node fragment, the claim that per-packet cost is unchanged by putting ranks on separate hosts is untested. Without the latency curve, the capacity in Section 5 implies a knee that was not observed, and nothing in this report characterises behaviour above it. Without the GPU arm, the constants of Equation 2 are unmeasured, and the argument that the bus decides the offload rests on the shape of the model rather than on this machine.

The admission model has no data to fit. The measurements were intended to support a control question: given the offered rate, the payload, the cipher and the network conditions, can the p99 latency be predicted well enough to decide whether to admit more traffic? The comparison that would matter is not whether a model fits, since a model fitted to a smooth curve always fits. It is whether a learned predictor beats a static threshold that admits traffic up to a fixed fraction of measured capacity, with splits taken by network condition rather than shuffled, so that neighbouring offered rates from one condition cannot leak across the boundary. That comparison needs the latency points the previous paragraph says are missing, so it is not reported.

One machine. Every number here comes from one host. The merge step refuses to combine fragments from different machines or builds, precisely because that would otherwise be invisible, but the consequence is that generality across hardware is argued from the structure of the model and not demonstrated.

9 Conclusion

Splitting the per-packet cost of an encrypted data plane into a fixed term and a per-byte term, and reporting their ratio, gives a quantity that survives the move from one machine to another where neither term alone does. On the host measured here, lr-aes-256-gcm gives $a = 6\,768$ cycles, $b = 1.869$ cycles per payload byte, and $s^* = 3\,621$ bytes against a maximum cariable payload of 1 432 bytes.

The same decomposition was the useful lens at the further boundaries this study could reach. At the kernel boundary, a submission path that issues no system calls measures directly what batching curves could previously only bound, and this run shows how sharply that measurement depends on the poller having a core of its own. At the isolation boundary, the tiers that built cost the same number of boundary crossings and differed by less than the run-to-run spread, which is a result about the host rather than about isolation. At the bus, the fixed cost and the transfer term together decide a batch size below which an accelerator cannot win regardless of how fast its kernel is.

Two methodological points carried more weight than expected. Measuring two independent implementations of each cipher separated what the algorithm costs from what one library's version costs, and it made the masked arm meaningful rather than a comparison between a software path and a hardware one. And writing every control's prediction down before running it turned the controls into something that could fail. One of them did, at first, because its threshold tested the wrong quantity, and fixing the control rather than the threshold is what made the instrumentation claim honest.

References

- [1] Jens Axboe. 2019. Efficient IO with `io_uring`. In *Kernel documentation*.
- [2] Tom Barbette, Cyril Soldani, and Laurent Mathy. 2018. Fast Userspace Packet Processing. In *ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*.
- [3] Jason A. Donenfeld. 2017. WireGuard: Next Generation Kernel Network Tunnel. In *Network and Distributed System Security Symposium (NDSS)*.
- [4] Morris Dworkin. 2007. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. Technical Report NIST Special Publication 800-38D. National Institute of Standards and Technology.
- [5] Shay Gueron and Michael E. Kounavis. 2010. Intel Carry-Less Multiplication Instruction and its Usage for Computing the GCM Mode. *Intel White Paper* (2010).

- [6] Torsten Hoeﬂer and Roberto Belli. 2015. Scientific Benchmarking of Parallel Computing Systems. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [7] Stephen Kent. 2005. *IP Encapsulating Security Payload (ESP)*. Technical Report RFC 4303. Internet Engineering Task Force.
- [8] Vlad Krasnov. 2017. It Takes Two to ChaCha (Poly). In *Cloudflare Blog*.
- [9] Antonis Manousis, Rahul Anand Sharma, Vyas Sekar, and Justine Sherry. 2020. Contention-Aware Performance Prediction for Virtualized Network Functions. In *ACM SIGCOMM*.
- [10] John D. McCalpin. 1995. Memory Bandwidth and Machine Balance in Current High Performance Computers. *IEEE Computer Society Technical Committee on Computer Architecture Newsletter* (1995), 19–25.
- [11] Yoav Nir and Adam Langley. 2018. *ChaCha20 and Poly1305 for IETF Protocols*. Technical Report RFC 8439. Internet Engineering Task Force.
- [12] Luigi Rizzo. 2012. netmap: A Novel Framework for Fast Packet I/O. In *USENIX Annual Technical Conference*.
- [13] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. In *Communications of the ACM*, Vol. 52. 65–76.
- [14] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2019. The True Cost of Containing: A gVisor Case Study. In *USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.